

MLOPS LAB GUIDE

Model Deployment, Real-Time Inference & Scalable Serving

Hands-On Lab Guide

Module: MLOps Foundations — Serving Layer

Cloud Stack: Amazon Web Services (AWS)

Audience Level: Intermediate (comfortable with Python, CLI, Docker)

How to Use This Guide

This guide contains three progressive, hands-on labs. Each lab builds on artifacts created in the previous one, so complete them in order. Every lab lists objectives, prerequisites, exact CLI/SDK commands, validation checks, common pitfalls, a cleanup checklist (to avoid unwanted AWS charges), and a challenge exercise for early finishers.

Cost Warning: SageMaker real-time endpoints bill per instance-hour even when idle. Always complete the Cleanup section at the end of each lab before leaving your AWS account unattended.

Labs in this Guide

1. Lab 1 — Model Deployment Strategies (Blue/Green & Canary releases)
2. Lab 2 — Real-Time Inference (low-latency synchronous serving via API)
3. Lab 3 — Scalable Model Serving (auto-scaling & multi-model endpoints)

Lab 1: Model Deployment Strategies

Duration	90–120 minutes
Cloud Services	Amazon SageMaker, IAM, S3, CloudWatch
Difficulty	Intermediate
Deliverable	Working AWS deployment + screenshot/output evidence submitted per instructions below

Learning Objectives

- Explain the difference between Blue/Green, Canary, and Shadow deployment strategies and when to use each
- Deploy two versions of a model behind a single SageMaker endpoint using weighted production variants
- Shift live traffic gradually from an old model version to a new one (canary rollout) without downtime
- Roll back instantly to the previous version if the new version underperforms

Prerequisites

- AWS account with permissions for SageMaker, IAM, S3, and CloudWatch
- AWS CLI v2 installed and configured (aws configure) — or use CloudShell / SageMaker Studio terminal
- Python 3.9+ with boto3 and sagemaker SDK: pip install boto3 sagemaker
- Basic familiarity with S3 buckets and IAM roles

Architecture Overview

You will host two versions of an XGBoost churn-prediction model (ModelV1 trained with max_depth=3, ModelV2 trained with max_depth=6) as two production variants on ONE SageMaker endpoint. SageMaker splits invocation traffic between variants according to a weight you control, letting you implement canary and blue/green patterns without spinning up separate load balancers.

Step-by-Step Instructions

Step 1: Set up your environment

Create a working S3 bucket and confirm your SageMaker execution role.

```
export BUCKET=mlops-lab-$(aws sts get-caller-identity --query Account --output text)
aws s3 mb s3://$BUCKET --region us-east-1
aws iam get-role --role-name SageMakerExecutionRole || \
  echo 'Create SageMakerExecutionRole in IAM console with AmazonSageMakerFullAccess first'
```

Step 2: Train two model versions

Use the SageMaker built-in XGBoost algorithm on the sample churn dataset. Save this as train_models.py and run it.

```
import boto3, sagemaker
from sagemaker.estimator import Estimator
from sagemaker import image_uris
```

```

sess = sagemaker.Session()
role = 'arn:aws:iam::<ACCOUNT_ID>:role/SageMakerExecutionRole'
bucket = '<BUCKET>'
image = image_uris.retrieve('xgboost', sess.boto_region_name, version='1.7-1')

def train(job_name, max_depth):
    est = Estimator(image, role, instance_count=1,
                    instance_type='ml.m5.large',
                    output_path=f's3://{bucket}/models/{job_name}',
                    sagemaker_session=sess)
    est.set_hyperparameters(objective='binary:logistic', num_round=50,
                           max_depth=max_depth, eta=0.2)
    est.fit({'train': f's3://{bucket}/data/train/train.csv',
            'validation': f's3://{bucket}/data/val/val.csv'},
          job_name=job_name)
    return est

model_v1 = train('churn-model-v1', max_depth=3)
model_v2 = train('churn-model-v2', max_depth=6)

```

No dataset handy?: Use the sample churn CSV from sagemaker-sample-files:
s3://sagemaker-sample-files/datasets/tabular/synthetic/churn.csv — split 80/20 into train.csv / val.csv and upload to the paths referenced above.

Step 3: Register both models with SageMaker

```

aws sagemaker create-model --model-name churn-v1 \
  --primary-container Image=<XGB_IMAGE_URI>,ModelDataUrl=s3://$BUCKET/models/churn-
model-v1/output/model.tar.gz \
  --execution-role-arn arn:aws:iam::<ACCOUNT_ID>:role/SageMakerExecutionRole

aws sagemaker create-model --model-name churn-v2 \
  --primary-container Image=<XGB_IMAGE_URI>,ModelDataUrl=s3://$BUCKET/models/churn-
model-v2/output/model.tar.gz \
  --execution-role-arn arn:aws:iam::<ACCOUNT_ID>:role/SageMakerExecutionRole

```

Step 4: Create an endpoint configuration with two weighted variants

This is the core of the canary pattern: 90% of traffic goes to the stable v1 variant, 10% goes to the new v2 variant.

```

aws sagemaker create-endpoint-config \
  --endpoint-config-name churn-canary-config \
  --production-variants '[
    {"VariantName": "Stable", "ModelName": "churn-v1", "InitialInstanceCount": 1,
      "InstanceType": "ml.m5.large", "InitialVariantWeight": 9},
    {"VariantName": "Canary", "ModelName": "churn-v2", "InitialInstanceCount": 1,
      "InstanceType": "ml.m5.large", "InitialVariantWeight": 1}
  ]'

```

Step 5: Deploy the endpoint

```

aws sagemaker create-endpoint \
  --endpoint-name churn-prod-endpoint \
  --endpoint-config-name churn-canary-config

# Poll until InService (takes 5-8 minutes)

```

```
aws sagemaker describe-endpoint --endpoint-name churn-prod-endpoint --query
EndpointStatus
```

Step 6: Send test traffic and confirm the split

```
import boto3, json, collections
rt = boto3.client('sagemaker-runtime')
counts = collections.Counter()
for i in range(200):
    resp = rt.invoke_endpoint(EndpointName='churn-prod-endpoint',
                              ContentType='text/csv', Body='0.5,1,0,42,1,0.2')
    counts[resp['InvokedProductionVariant']] += 1
print(counts)  # expect roughly Stable ~180, Canary ~20
```

Cross-check in the AWS Console: CloudWatch > Metrics > SageMaker > Endpoint Metrics, filter by VariantName, and graph Invocations and ModelLatency for Stable vs Canary side by side.

Step 7: Progressively shift traffic (canary rollout)

If Canary metrics look healthy (latency, error rate, business KPI), increase its weight in stages:

```
# 50/50
aws sagemaker update-endpoint-weights-and-capacities \
  --endpoint-name churn-prod-endpoint \
  --desired-weights-and-capacities VariantName=Stable,DesiredWeight=5
VariantName=Canary,DesiredWeight=5

# 100% to the new version (full cutover – completes the Blue/Green swap)
aws sagemaker update-endpoint-weights-and-capacities \
  --endpoint-name churn-prod-endpoint \
  --desired-weights-and-capacities VariantName=Stable,DesiredWeight=0
VariantName=Canary,DesiredWeight=10
```

Step 8: Practice an instant rollback

Simulate discovering a regression in v2 immediately after the 50/50 shift, and roll fully back to v1 with a single API call:

```
aws sagemaker update-endpoint-weights-and-capacities \
  --endpoint-name churn-prod-endpoint \
  --desired-weights-and-capacities VariantName=Stable,DesiredWeight=10
VariantName=Canary,DesiredWeight=0
```

Blue/Green vs Canary: What you just built IS both patterns — the only difference is how fast you move the weight from 0 to 100. A true Blue/Green swap jumps straight from 100/0 to 0/100 in one step; a Canary rollout pauses at intermediate weights (e.g., 90/10 → 50/50 → 0/100) to observe metrics at each stage before continuing.

Validation Checklist

- Endpoint status shows InService for both variants
- 200-request test shows traffic split matching configured weights ($\pm 10\%$)
- CloudWatch shows separate ModelLatency and Invocations graphs per VariantName
- Rollback command successfully returns 100% of traffic to Stable within seconds

Troubleshooting

Issue	Likely Cause / Fix
Endpoint stuck in Creating	Check CloudWatch Logs group /aws/sagemaker/Endpoints/<name> for container startup errors; verify ModelDataUrl path and image URI region match
ValidationException on create-model	The execution role ARN must trust sagemaker.amazonaws.com and the S3 path must exist and be in the same region
Traffic split looks wrong at low volume	Weighted routing is probabilistic — run 500+ requests for the ratio to converge closely to configured weights
AccessDenied calling invoke-endpoint	Attach AmazonSageMakerFullAccess (or a scoped sagemaker:InvokeEndpoint policy) to the caller's IAM identity

Challenge Exercise

Add a third variant ("Shadow") that receives a mirrored copy of production traffic but whose responses are never returned to the caller — log its predictions to CloudWatch only, and compare them offline against the Stable variant's predictions to validate a new model with zero user-facing risk.

Cleanup

```
aws sagemaker delete-endpoint --endpoint-name churn-prod-endpoint
aws sagemaker delete-endpoint-config --endpoint-config-name churn-canary-config
aws sagemaker delete-model --model-name churn-v1
aws sagemaker delete-model --model-name churn-v2
```

Lab 2: Real-Time Inference

Duration	75–90 minutes
Cloud Services	Amazon SageMaker, IAM, S3, CloudWatch, Lambda, API Gateway, Application Auto Scaling
Difficulty	Intermediate
Deliverable	Working AWS deployment + screenshot/output evidence submitted per instructions below

Learning Objectives

- Deploy a persistent, low-latency SageMaker real-time inference endpoint
- Expose the endpoint publicly through API Gateway + Lambda so any HTTP client can consume it
- Capture request/response payloads automatically for later monitoring and debugging
- Measure and interpret p50 / p95 / p99 latency for a synchronous inference API

Prerequisites

- Completion of Lab 1 (a deployed churn model, or reuse churn-v1)
- IAM permissions for Lambda and API Gateway in addition to Lab 1's permissions
- Python requests library and curl installed locally

Architecture Overview

Client → API Gateway (REST API, public HTTPS endpoint) → Lambda (thin proxy) → SageMaker real-time endpoint (invoke_endpoint) → response returned synchronously to the client, typically within tens to low-hundreds of milliseconds.

Step-by-Step Instructions

Step 1: Deploy a single-variant real-time endpoint

If Lab 1's endpoint was deleted, redeploy a simple single-variant endpoint for churn-v1:

```
aws sagemaker create-endpoint-config \
  --endpoint-config-name churn-rt-config \
  --production-variants VariantName=Primary,ModelName=churn-
v1,InitialInstanceCount=1,InstanceType=ml.m5.large,InitialVariantWeight=1

aws sagemaker create-endpoint --endpoint-name churn-rt-endpoint --endpoint-config-
name churn-rt-config
```

Step 2: Write the Lambda proxy function

Create lambda_function.py:

```
import boto3, json
rt = boto3.client('sagemaker-runtime')

def handler(event, context):
    body = json.loads(event['body'])
```

```

csv_row = ','.join(str(v) for v in body['features'])
resp = rt.invoke_endpoint(
    EndpointName='churn-rt-endpoint',
    ContentType='text/csv',
    Body=csv_row,
)
score = resp['Body'].read().decode('utf-8').strip()
return {
    'statusCode': 200,
    'headers': {'Content-Type': 'application/json'},
    'body': json.dumps({'churn_probability': float(score)})
}

```

Step 3: Package and deploy the Lambda function

```

zip function.zip lambda_function.py

aws iam create-role --role-name lambda-sagemaker-role \
  --assume-role-policy-document file://trust-policy.json
aws iam attach-role-policy --role-name lambda-sagemaker-role \
  --policy-arn arn:aws:iam::aws:policy/AmazonSageMakerFullAccess
aws iam attach-role-policy --role-name lambda-sagemaker-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

aws lambda create-function --function-name invoke-churn-model \
  --runtime python3.11 --handler lambda_function.handler \
  --zip-file fileb://function.zip \
  --role arn:aws:iam::<ACCOUNT_ID>:role/lambda-sagemaker-role \
  --timeout 10 --memory-size 256

```

Step 4: Create a public API Gateway REST endpoint

Fastest path: use the API Gateway console → Create API → HTTP API → Add integration (Lambda: invoke-churn-model) → Add route POST /predict → Deploy to stage 'prod'. Or via CLI:

```

aws apigatewayv2 create-api --name churn-api --protocol-type HTTP \
  --target arn:aws:lambda:us-east-1:<ACCOUNT_ID>:function:invoke-churn-model

aws lambda add-permission --function-name invoke-churn-model \
  --statement-id apigw-invoke --action lambda:InvokeFunction \
  --principal apigateway.amazonaws.com

```

Step 5: Test the live API

```

curl -X POST https://<api-id>.execute-api.us-east-1.amazonaws.com/predict \
  -H 'Content-Type: application/json' \
  -d '{"features": [0.5, 1, 0, 42, 1, 0.2]}'

# Expected response
# {"churn_probability": 0.0731}

```

Step 6: Turn on SageMaker Data Capture

Data Capture logs every request/response pair to S3, which you will use for monitoring in Lab 3 of the next guide.

```

aws sagemaker update-endpoint \

```

```
--endpoint-name churn-rt-endpoint \
--endpoint-config-name churn-rt-config-with-capture

# churn-rt-config-with-capture must be created with:
# "DataCaptureConfig": {
#   "EnableCapture": true,
#   "InitialSamplingPercentage": 100,
#   "DestinationS3Uri": "s3://<BUCKET>/datacapture",
#   "CaptureOptions": [{"CaptureMode": "Input"}, {"CaptureMode": "Output"}]
# }
```

Step 7: Measure latency

Save as `load_test.py` and run it:

```
import time, statistics, requests

URL = 'https://<api-id>.execute-api.us-east-1.amazonaws.com/predict'
latencies = []
for _ in range(100):
    t0 = time.time()
    requests.post(URL, json={'features':[0.5,1,0,42,1,0.2]})
    latencies.append((time.time() - t0) * 1000)

latencies.sort()
print('p50:', latencies[49], 'ms')
print('p95:', latencies[94], 'ms')
print('p99:', latencies[98], 'ms')
```

Validation Checklist

- curl request returns a valid JSON prediction with HTTP 200
- S3 bucket datacapture/ prefix contains JSON-lines capture files after a few requests
- Load test script prints p50/p95/p99 and p95 is under ~300 ms for ml.m5.large

Troubleshooting

Issue	Likely Cause / Fix
502 from API Gateway	Check CloudWatch Logs for the Lambda function — usually a Python exception or a timeout shorter than the endpoint's cold response time
Lambda 'AccessDeniedException' on invoke_endpoint	The Lambda execution role is missing sagemaker:InvokeEndpoint permission
High p95/p99 with low p50	Classic cold-start / GC pause pattern; consider provisioned concurrency on Lambda or a larger instance type for the endpoint
No files appearing in datacapture/	Confirm the endpoint config actually has DataCaptureConfig attached — updating an endpoint requires a NEW endpoint-config resource, you cannot edit one in place

Challenge Exercise

Add a second Lambda route (/predict-async) that instead calls SageMaker Asynchronous Inference (CreateEndpoint with AsyncInferenceConfig) and returns an S3 pointer immediately. Compare when you would choose real-time vs. asynchronous vs. batch inference for a given SLA.

Cleanup

```
aws apigatewayv2 delete-api --api-id <api-id>
aws lambda delete-function --function-name invoke-churn-model
aws sagemaker delete-endpoint --endpoint-name churn-rt-endpoint
aws sagemaker delete-endpoint-config --endpoint-config-name churn-rt-config-with-capture
```

Lab 3: Scalable Model Serving

Duration	90 minutes
Cloud Services	Amazon SageMaker, IAM, S3, CloudWatch, Lambda, API Gateway, Application Auto Scaling
Difficulty	Intermediate
Deliverable	Working AWS deployment + screenshot/output evidence submitted per instructions below

Learning Objectives

- Configure target-tracking auto scaling on a SageMaker real-time endpoint
- Trigger and observe a scale-out event under simulated load
- Host multiple models cost-efficiently on one endpoint using a Multi-Model Endpoint (MME)
- Compare real-time, serverless, and multi-model hosting on cost and latency

Prerequisites

- Completion of Labs 1–2
- IAM permission for Application Auto Scaling (application-autoscaling:*)

Architecture Overview

Part A wires the churn-rt-endpoint from Lab 2 to Application Auto Scaling, which watches SageMakerVariantInvocationsPerInstance and adds/removes instances to hold that metric near a target value. Part B replaces single-model hosting with a Multi-Model Endpoint that lazily loads model artifacts from a shared S3 prefix, letting you serve many models from one fleet of instances.

Part A — Auto Scaling a Real-Time Endpoint

Step 1: Register the endpoint as a scalable target

```
aws application-autoscaling register-scalable-target \
  --service-namespace sagemaker \
  --resource-id endpoint/churn-rt-endpoint/variant/Primary \
  --scalable-dimension sagemaker:variant:DesiredInstanceCount \
  --min-capacity 1 --max-capacity 4
```

Step 2: Attach a target-tracking scaling policy

```
aws application-autoscaling put-scaling-policy \
  --service-namespace sagemaker \
  --resource-id endpoint/churn-rt-endpoint/variant/Primary \
  --scalable-dimension sagemaker:variant:DesiredInstanceCount \
  --policy-name churn-target-tracking \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration '{
    "TargetValue": 50.0,
    "PredefinedMetricSpecification": {
      "PredefinedMetricType": "SageMakerVariantInvocationsPerInstance"
    }
  }'
```

```

    },
    "ScaleInCooldown": 300,
    "ScaleOutCooldown": 60
  }'

```

This tells SageMaker: keep invocations-per-instance-per-minute near 50; add an instance quickly (60s cooldown) when load rises, remove instances cautiously (300s cooldown) when load falls.

Step 3: Generate load to trigger a scale-out

```

import concurrent.futures, boto3
rt = boto3.client('sagemaker-runtime')

def call(_):
    rt.invoke_endpoint(EndpointName='churn-rt-endpoint',
        ContentType='text/csv', Body='0.5,1,0,42,1,0.2')

with concurrent.futures.ThreadPoolExecutor(max_workers=50) as ex:
    list(ex.map(call, range(5000)))

```

Step 4: Observe the scaling event

- SageMaker console → Endpoints → churn-rt-endpoint → watch Instance count rise from 1 toward 4
- CloudWatch → Alarms — Application Auto Scaling creates two managed alarms automatically; inspect their state transitions
- CloudWatch Logs → confirm new instances register and pass health checks

Part B — Multi-Model Endpoint (MME)

Step 5: Stage multiple model artifacts under one S3 prefix

```

aws s3 cp s3://$BUCKET/models/churn-model-v1/output/model.tar.gz \
s3://$BUCKET/mme-models/churn-v1.tar.gz
aws s3 cp s3://$BUCKET/models/churn-model-v2/output/model.tar.gz \
s3://$BUCKET/mme-models/churn-v2.tar.gz

```

Step 6: Create the multi-model container and endpoint

```

aws sagemaker create-model --model-name churn-mme \
--primary-container
Image=<XGB_IMAGE_URI>,Mode=MultiModel,ModelDataUrl=s3://$BUCKET/mme-models/ \
--execution-role-arn arn:aws:iam::<ACCOUNT_ID>:role/SageMakerExecutionRole

aws sagemaker create-endpoint-config --endpoint-config-name churn-mme-config \
--production-variants VariantName=Primary,ModelName=churn-
mme,InitialInstanceCount=1,InstanceType=ml.m5.large,InitialVariantWeight=1

aws sagemaker create-endpoint --endpoint-name churn-mme-endpoint --endpoint-config-
name churn-mme-config

```

Step 7: Invoke specific models on the shared endpoint

```

import boto3
rt = boto3.client('sagemaker-runtime')

resp_v1 = rt.invoke_endpoint(EndpointName='churn-mme-endpoint',

```

```

    TargetModel='churn-v1.tar.gz', ContentType='text/csv', Body='0.5,1,0,42,1,0.2')
resp_v2 = rt.invoke_endpoint(EndpointName='churn-mme-endpoint',
    TargetModel='churn-v2.tar.gz', ContentType='text/csv', Body='0.5,1,0,42,1,0.2')

print(resp_v1['Body'].read(), resp_v2['Body'].read())

```

How MME saves cost: The first invocation of a given TargetModel triggers a lazy load from S3 onto the instance (adds latency once); subsequent calls hit an in-memory LRU cache. Ten rarely-used models can share one small fleet instead of ten always-on endpoints.

Part C — Compare Hosting Options

Real-time endpoint	Always-on instance(s); lowest, most predictable latency; pay per instance-hour regardless of traffic
Serverless inference	No idle cost, auto-provisions on request; cold starts add latency; good for spiky/low-traffic workloads
Multi-model endpoint	One fleet serves many models; great for large model catalogs with uneven traffic; first call per model is slower
Async inference	For large payloads / long-running inference; returns immediately with a result pointer, not a live response

Validation Checklist

- Instance count visibly increases during the load test and later scales back in
- Both churn-v1.tar.gz and churn-v2.tar.gz respond correctly via TargetModel on the same MME endpoint
- You can articulate, in one sentence each, when to choose real-time vs serverless vs multi-model vs async

Troubleshooting

Issue	Likely Cause / Fix
Scaling never triggers	5,000 requests may complete before CloudWatch's 1-minute metric period aggregates; rerun the load generator for 3-5 minutes continuously
ModelError: model not found (MME)	TargetModel must exactly match the S3 object key (including .tar.gz) under the ModelDataUrl prefix
First MME request times out	Increase the endpoint's ContainerStartupHealthCheckTimeoutInSeconds or verify the artifact size isn't unusually large for ml.m5.large memory

Challenge Exercise

Convert churn-rt-endpoint to Serverless Inference (ProductionVariants[].ServerlessConfig) instead of an always-on instance, repeat the p95 latency measurement from Lab 2, and write two sentences comparing cost and latency trade-offs versus the provisioned real-time endpoint.

Final Cleanup — Run This Before Ending the Session

```
aws application-autoscaling deregister-scalable-target \
```

```
--service-namespace sagemaker --resource-id  
endpoint/churn-rt-endpoint/variant/Primary \  
--scalable-dimension sagemaker:variant:DesiredInstanceCount  
aws sagemaker delete-endpoint --endpoint-name churn-mme-endpoint  
aws sagemaker delete-endpoint-config --endpoint-config-name churn-mme-config  
aws sagemaker delete-model --model-name churn-mme  
aws sagemaker delete-endpoint --endpoint-name churn-rt-endpoint  
aws sagemaker delete-endpoint-config --endpoint-config-name churn-rt-config  
aws s3 rb s3://$BUCKET --force # only if no longer needed for the next guide
```